ASSIGNMENT 6

Name: Rohit Kailas Deore
Roll No: 11
Batch: T1
Class: TE-IT

[1]:
```
# Import required libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

print("Libraries imported successfully.")
```

```
Libraries imported successfully.
```

1. Load the California Housing Dataset

[2]:
```
# Load the dataset from a public CSV link (no kagglehub required)
url = "https://raw.githubusercontent.com/ageron/handson-ml2/master/datasets/
  ↪housing/housing.csv"
df = pd.read_csv(url)

print("First five rows:")
display(df.head())
print("Dataset shape:", df.shape)
```

```
First five rows:
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	\
0	-122.23	37.88	41.0	880.0	129.0	
1	-122.22	37.86	21.0	7099.0	1106.0	
2	-122.24	37.85	52.0	1467.0	190.0	
3	-122.25	37.85	52.0	1274.0	235.0	
4	-122.25	37.85	52.0	1627.0	280.0	

	population	households	median_income	median_house_value	ocean_proximity
0	322.0	126.0	8.3252	452600.0	NEAR BAY
1	2401.0	1138.0	8.3014	358500.0	NEAR BAY
2	496.0	177.0	7.2574	352100.0	NEAR BAY
3	558.0	219.0	5.6431	341300.0	NEAR BAY
4	565.0	259.0	3.8462	342200.0	NEAR BAY

```
Dataset shape: (20640, 10)
```

2. Prepare the Data

[3]:
```
# Fill missing values in total_bedrooms
df["total_bedrooms"] = df["total_bedrooms"].fillna(df["total_bedrooms"].median())

# Select three numerical input features and the target
X = df[["median_income", "housing_median_age", "total_rooms"]]
y = df["median_house_value"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

```
Training data: (16512, 3)
Testing data: (4128, 3)
```

3. Apply Linear Regression

[4]:
```
linear_model = LinearRegression()
linear_model.fit(X_train, y_train)

y_pred_linear = linear_model.predict(X_test)

linear_mse = mean_squared_error(y_test, y_pred_linear)
linear_mae = mean_absolute_error(y_test, y_pred_linear)
linear_r2 = r2_score(y_test, y_pred_linear)

print("Linear Regression Results")
print("MSE:", round(linear_mse, 2))
print("MAE:", round(linear_mae, 2))
print("R2 Score:", round(linear_r2, 4))
```

```
Linear Regression Results
MSE: 6563178660.01
MAE: 60348.04
R2 Score: 0.4992
```

4. Apply Polynomial Regression

[5]:
```
# Convert input features into polynomial features of degree 2
poly = PolynomialFeatures(degree=2)
X_train_poly = poly.fit_transform(X_train)
X_test_poly = poly.transform(X_test)

poly_model = LinearRegression()
```

```
poly_model.fit(X_train_poly, y_train)

y_pred_poly = poly_model.predict(X_test_poly)

poly_mse = mean_squared_error(y_test, y_pred_poly)
poly_mae = mean_absolute_error(y_test, y_pred_poly)
poly_r2 = r2_score(y_test, y_pred_poly)

print("Polynomial Regression Results")
print("MSE:", round(poly_mse, 2))
print("MAE:", round(poly_mae, 2))
print("R2 Score:", round(poly_r2, 4))
```

```
Polynomial Regression Results
MSE: 6920280005.02
MAE: 61993.36
R2 Score: 0.4719
```

5. Compare Both Models

[6]:
```
comparison = pd.DataFrame({
    "Model": ["Linear Regression", "Polynomial Regression"],
    "MSE": [linear_mse, poly_mse],
    "MAE": [linear_mae, poly_mae],
    "R2 Score": [linear_r2, poly_r2]
})

display(comparison.round(3))

# The model with the lower MSE is better on this test set
if linear_mse < poly_mse:
    print("Better model based on MSE: Linear Regression")
else:
    print("Better model based on MSE: Polynomial Regression")
```

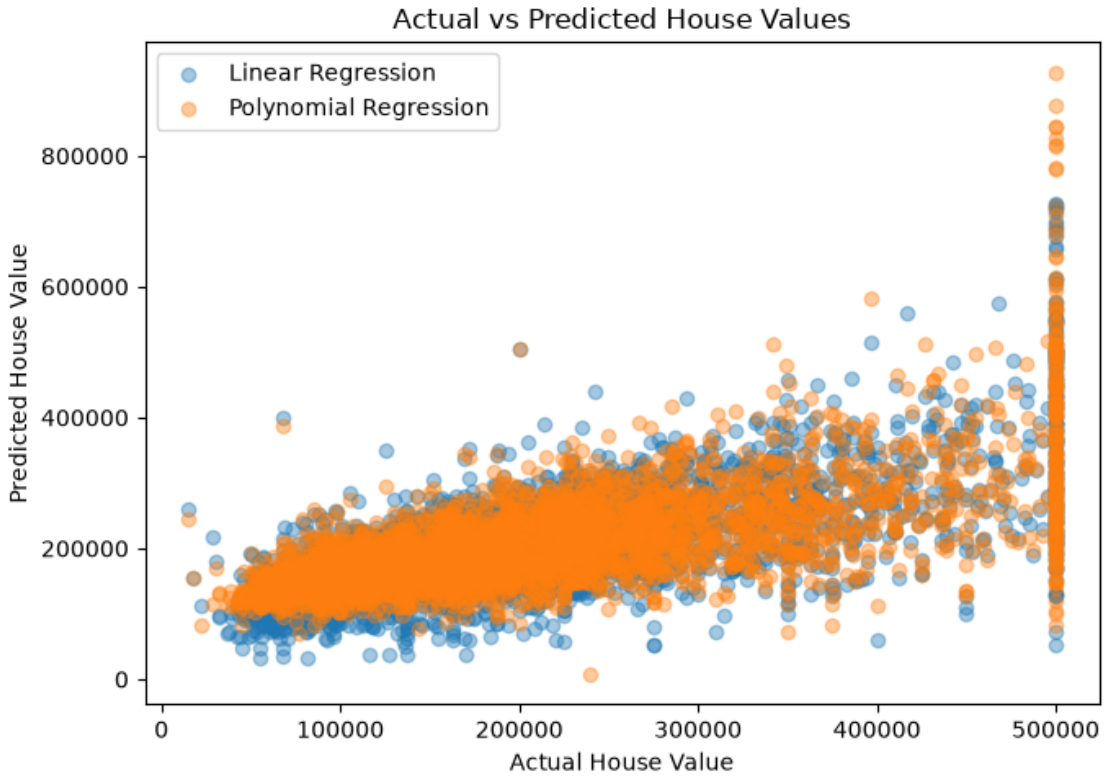
	Model	MSE	MAE	R2 Score
0	Linear Regression	6.563179e+09	60348.038	0.499
1	Polynomial Regression	6.920280e+09	61993.358	0.472

```
Better model based on MSE: Linear Regression
```

6. Visualize Actual and Predicted Values

[7]:
```
# Compare actual house values with predictions from both models
plt.figure(figsize=(7, 5))
plt.scatter(y_test, y_pred_linear, alpha=0.4, label="Linear Regression")
plt.scatter(y_test, y_pred_poly, alpha=0.4, label="Polynomial Regression")
plt.xlabel("Actual House Value")
```

```
plt.ylabel("Predicted House Value")
plt.title("Actual vs Predicted House Values")
plt.legend()
plt.tight_layout()
plt.show()
```



Conclusion

Linear Regression and Polynomial Regression were implemented on the California Housing dataset. Both models were evaluated using Mean Squared Error (MSE), Mean Absolute Error (MAE), and R^2 score. The comparison table and graph help identify which model performs better on the test data.